

EarsForYou: Frontend-Backend Integration & Test Credentials

This document outlines the recent improvements made to the communication layer between the React/Next.js frontend and the Spring Boot backend, ensuring a secure, scalable, and developer-friendly architecture.

It also includes the database-seeded test accounts for rapid UI development.

1. Test Account Credentials

The backend now features a `TestAccountSeeder` that runs on server startup. It guarantees these accounts are always available, allowing developers to bypass the OTP email flow during UI testing.

Standard User (Main App)

- **Email:** `test@earsforyou.com`
- **Password:** `Password123!`

Super Admin (Admin Dashboard)

- **Email:** `admin@earsforyou.com`
- **Password:** `Admin123!`

2. API Communication Best Practices Implemented

Based on recent collaboration between the frontend and backend teams, the following integration standards have been applied to the API contract:

2.1 Standardized Error Handling

- **The Issue:** The API previously returned empty `403` or `500` response bodies, making it impossible for the frontend to display accurate toast notifications to the user (e.g., distinguishing a bad password from a suspended account).



- **The Solution:** A `GlobalExceptionHandler` was implemented. Every error response across the entire application now returns a consistent JSON structure: `{ "message": "Human-readable error description" }`.
- **Status Codes:**
 - 400 (Bad Request): Validation errors or business logic violations.
 - 401 (Unauthorized): Invalid credentials or expired tokens.
 - 403 (Forbidden): Valid token, but lacking required role permissions.
 - 429 (Too Many Requests): Rate limit exceeded.

2.2 Secure URL Payloads (Query String Mitigation)

- **The Issue:** Sensitive operations, like password recovery and OTP resending, were passing emails and OTPs as URL query parameters (`?email=...&otp=...`). This risked exposing sensitive data in browser histories, proxy logs, and analytics tracking.
- **The Solution:** All sensitive endpoints have been refactored to consume `@RequestBody` JSON payloads.
 - *Example Payload:* `{ "email": "user@example.com", "otp": "123456" }`

2.3 Timezone Independence

- **The Issue:** The server (running in UTC on Render) returned raw timestamps, causing the frontend's "Did I check in today?" logic to fail for users in different timezones (e.g., Nigeria/WAT).
- **The Solution:**
 - **Dashboard:** The server now performs the timezone calculation internally based on the user's registered country, returning a simple `"loggedToday": true` boolean to the frontend.
 - **Charts:** Chart data now returns standard `ISO_LOCAL_DATE` strings (e.g., `"2026-08-06"`) rather than pre-formatted strings, allowing the frontend to handle localization natively.

2.4 Secure Cross-Domain Cookies

- **The Implementation:** To support silent token refreshing without exposing tokens to XSS attacks, the backend utilizes `HttpOnly` cookies for the `refresh_token` .
- **Dynamic Hardening:** The cookie generator dynamically applies `Secure=true` and `SameSite=None` when deployed to production, satisfying strict browser requirements for cross-domain communication between Vercel and Render. *(Note: The frontend must include `withCredentials: true` in all Axios requests to transmit these cookies).*

2.5 Health Check & "Cold Start" Mitigation

- **The Issue:** Free-tier cloud hosting (Render) sleeps after inactivity, resulting in 60-second cold starts.
- **The Solution:** Two public endpoints were whitelisted to allow the frontend to safely ping the server and display a "Connecting..." UI while it wakes up:
 - `GET /actuator/health` (Returns `{"status": "UP"}`)
 - `GET /` (Returns `{"status": "online", "message": "..."}`)

2.6 CORS Management

- **The Implementation:** The `CorsConfig` was updated to explicitly trust `http://localhost:3000` (Next.js default) alongside the standard Vite ports for local development. Production allowed origins are managed dynamically via the `APP_FRONTEND_URLS` environment variable on Render.